

Cấu trúc dữ liệu

Thực hành 1 - Độ phức tạp của thuật toán

Bài tập 1: Xác định số hạng có bậc cao nhất và độ phức tạp của thuật toán:

T(n)	Số hạng có bậc cao nhất	O(n)
$5 + 0.001n^3 + 0.025n$	$0.001n^3$	$O(n^3)$
$500 + 100n^{1.5} + 50n\log_{10}n$	$100n^{1.5}$	$O(n^{1.5})$
$n^2\log_2n + n(\log_2n)^2$	$n^2\log_2n$	$O(n^2\log n)$
$n\log_3n + n\log_2n$	$n\log_3n, n\log_2n$	$O(n\log n)$
$3\log_8n + \log_2\log_2\log_2n$	$3\log_8n$	$O(\log n)$
$100n\log_3n + n^3 + 100n$	n^3	$O(n^3)$

Bài tập 2: Xác định độ phức tạp của các thuật toán sau:

- 1) for (i = 0; i < n; i++) $\Rightarrow O(n)$
- 2) for (i = 0; i < n; i=i+2) $\Rightarrow O(n)$
- 3) for (i = 1; i < n; i=i*2) $\Rightarrow O(\log n)$
- 4) for (i = 1; i < n; i=i*3) $\Rightarrow O(\log n)$
- 5) for (i = n; i > 1; i=i/2) $\Rightarrow O(\log n)$

Bài tập 3: Xác định độ phức tạp của các thuật toán sau:

```

int result = 0;
for(int i = 0, i < n; i++){
    if(array[i] < 10){
        result = result + array[i];
    }
    for(int j = 0; j < n, j++){
        print(array[i])
    }
}

```

Độ phức tạp của thuật toán là $O(n^2)$

Bài tập 4: Thuật toán A có thời gian $TA(n) = c_A n \log_2 n$ và thuật toán B có thời gian $TB(n) = c_B n^2$ với kích thước dữ liệu đầu vào là n . Tìm thuật toán tối ưu để xử lý dữ liệu với kích thước $n = 2^{20}$, biết rằng thuật toán A cần 10 us và thuật toán B cần 1 us để xử lý 1024 dữ liệu.

Tốc độ xử lý của thuật toán A là

$$\frac{1024}{10} = 102.4 \text{ [dữ liệu/ microsecond]}$$

Tốc độ xử lý của thuật toán B là

$$\frac{1024}{1} = 1024 \text{ [dữ liệu/ microsecond]}$$

Thời gian để thuật toán A xử lý 2^{20} dữ liệu là

$$\frac{2^{20}}{102.4} = 10240 \text{ [microsecond]}$$

Thời gian để thuật toán B xử lý 2^{20} dữ liệu là

$$\frac{2^{20}}{1024} = 1024 \text{ [microsecond]}$$

Vậy thuật toán B phù hợp hơn để xử lý 2^{20} dữ liệu

Bài tập 5: Thuật toán A có thời gian $TA(n) = 5n \log_{10} n$ và thuật toán B có thời gian $TB(n) = 25n$ với kích thước dữ liệu đầu vào là n . Thuật toán nào tốt hơn trong dựa trên phân tích Big-Oh. Với dữ liệu đầu vào như thế nào thì thuật toán đó có thể xử lý trong thời gian ngắn hơn thuật toán còn lại?

Với dữ liệu có kích thước là n

Thời gian xử lý của thuật toán A là $5n \log_{10} n$ [microsecond]

Thời gian xử lý của thuật toán B là $25n$ [microsecond]

Dựa trên phân tích Big-Oh thì thuật toán B tốt hơn thuật toán A

Cụ thể hơn, thuật toán B sẽ tốt hơn thuật toán A khi

$$25n < 5n \log_{10} n$$

$$\Leftrightarrow 5 < \log_{10} n$$

$$\Leftrightarrow n > 10^5$$

Bài 6: Xác định độ phức tạp của thuật toán sau là đúng hay sai:

- (a) $2n = O(n)$ => Đúng
- (b) $n^2 = O(n)$ => Sai
- (c) $n^2 = O(n \log n)$ => Sai
- (d) $n \log n = O(n^2)$ => Đúng
- (e) $3^n = O(2^n)$ => Sai
- (f) $3^n = 2^{O(n)}$ => Đúng
- (g) $2^{2n} = O(2^{2n})$ => Đúng

Giải thích câu f) $3^n = (3^{\log_2 2})^n = (2^{\log_2 3})^n = 2^{n \log_2 3}$

Vì $\log_2 3$ là hằng số nên $n \log_2 3 = O(n)$, suy ra $3^n = 2^{O(n)}$

Bài 7:

Một mảng chưa được sắp xếp với kích thước n bao gồm các số nguyên từ 1 đến n+1, với 1 phần tử bị thiếu đi, Hãy viết thuật toán có độ phức tạp $O(n)$ để tìm phần tử bị thiếu đó nhưng không tốn thêm bộ nhớ.

```
// Tìm phần tử bị thiếu trong mảng
int getMissingNumber(int arr[], int n)
{
    // kích thước của mảng là n+1
    int m = n + 1;
    // tổng các số nguyên từ 1 đến n+1
    int total = m*(m + 1)/2;
    // tính tổng những phần tử tồn tại trong mảng
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += arr[i];
    }
    // phần tử bị thiếu sẽ bằng total - sum
    return total - sum;
}
```

Câu 8:

Đánh giá thuật toán tìm phần tử nhỏ nhất trong mảng $A[0, \dots, n]$. Với 1 biến tạm là temp để giữ giá trị nhỏ nhất hiện tại. Bắt đầu từ $A[0]$, biến temp sẽ được so sánh với $A[1], A[2], A[3], A[4], \dots, A[n]$. Khi $A[i] < \text{temp}$, thì $\text{temp} = A[i]$. Việc gán $\text{temp} = A[i]$ sẽ được thực hiện bao nhiêu lần?

Trong trường hợp tệ nhất, khi mảng được sắp xếp giảm dần, thì việc gán $\text{temp} = A[i]$ sẽ thực hiện n lần (bao gồm cả việc gán $\text{temp} = A[0]$, nhưng với mảng sắp xếp tăng dần thì chỉ cần 1 lần (trường hợp tốt nhất đó là $A[0]$ đã là giá trị nhỏ nhất rồi). Ở trường hợp trung bình sẽ là $n/2$ lần.